Module 2 — Class 4 Assignment: Pandas Basics

Type-it-yourself exercise

UN AI/ML Mentorship

Module 2 — Class 4 Assignment: Pandas Basics

Topic: Pandas — DataFrames, filtering, grouping, sorting

Goal: Open a **fresh Google Colab notebook** and type the code yourself by following the steps below. Do NOT copy-paste from the working notebook. Typing it yourself is how you learn the syntax.

How to submit

1. Open Google Colab → File → New Notebook.
2. Type the code from each step below. Add a short `#` comment on EVERY cell so we know you understand it.
3. Save the notebook as `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`.
4. Push it to your team repo at `module-<X>/class_<Y>/submissions/<YourName>/`.

Step 0 — Load the data

What to do: import pandas and load the Telco customer dataset.

```
import pandas as pd
url = 'https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Cu
df = pd.read_csv(url)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
print('Shape:', df.shape)
```

Expected output: `Shape: (7043, 21)`

Step 1 — See column info

What to do: Print the dtypes of each column.

Type this in a new cell:

```
df.info()
```

Expected output:

```
RangeIndex: 7043 entries, ...\n(21 columns listed with dtypes)
```

Step 2 — Statistics for numeric columns

What to do: Use describe() to see min/max/mean of every numeric column.

Type this in a new cell:

```
df.describe().round(2)
```

Expected output:

```
A table with SeniorCitizen, tenure, MonthlyCharges, TotalCharges rows.
```

Step 3 — Count unique values per column

What to do: Show how many unique values are in each column.

Type this in a new cell:

```
df.nunique()
```

Expected output:

```
customerID    7043\ngender           2\ntenure          73\n...
```

Step 4 — Filter — customers paying more than $80/month

What to do: Keep only rows where MonthlyCharges > 80.

Type this in a new cell:

```
high_pay = df[df['MonthlyCharges'] > 80]
print('High-paying customers:', len(high_pay))
high_pay.head()
```

Expected output:

```
High-paying customers: 2756
```

Step 5 — Sort — top 10 by tenure

What to do: Sort by tenure descending and take the first 10 rows.

Type this in a new cell:

```
top10 = df.sort_values('tenure', ascending=False).head(10)
top10[['customerID', 'tenure', 'Contract', 'MonthlyCharges']]
```

Expected output:

```
10 rows, all with tenure = 72.
```

Step 6 — Groupby — avg MonthlyCharges per PaymentMethod

What to do: Group by PaymentMethod and compute mean MonthlyCharges.

Type this in a new cell:

```
df.groupby('PaymentMethod')['MonthlyCharges'].mean().round(2)
```

Expected output:

```
PaymentMethod\nBank transfer (automatic)   67.21\nCredit card (automatic)    66.39\nElectronic
```

Step 7 — Groupby with two stats

What to do: Group by Churn and compute mean AND count of TotalCharges.

Type this in a new cell:

```
df.groupby('Churn').agg({'TotalCharges': ['mean', 'count']}).round(2)
```

Expected output:

```
TotalCharges            mean   count\nChurn\nNo                  2549.91    5174\nYes
```

Step 8 — Crosstab — Contract vs Churn

What to do: Use pd.crosstab to see how many people from each Contract churned.

Type this in a new cell:

```
pd.crosstab(df['Contract'], df['Churn'])
```

Expected output:

```
Churn              No   Yes\nContract\nMonth-to-month    2220  1655\nOne year          1307   1
```

Checklist before you submit

- ☐ All cells run without error
- ☐ Every code cell has a `#` comment in your own words
- ☐ Outputs are visible (you saved AFTER running)
- ☐ File name is correct: `Module<X>_Class<Y>_<YourName>_Assignment.ipynb`

Deadline: before next class.